Implicit Return Type and Allowing Anonymous Types as Return Values

Document: P0536R0
Date: 2017-02-03
Project: ISO/IEC JTC1 SC22 WG21 Programming Language C++
Audience: Evolution Working Group
Author: Matthew Woehlke (mwoehlke.floss@gmail.com)

Abstract

This proposal recommends the relaxing of [dcl.fct]¶11; specifically, the prohibition of defining (anonymous) types as return values, and adding a mechanism that simplifies naming the return type of a previously declared function. These features were previously proposed separately, as P0222 and P0224.

Contents

Rationale

The concept of multiple return values is well known. At present, however, C++ lacks a good mechanism for implementing the same. `std::tuple` is considered clunky by many and, critically, creates sub-optimal API by virtue of the returned values being unnamed, forcing developers to rely on supplemental documentation to explain their purpose. Aggregates represent an improvement, being self-documenting, but the need to provide external definitions of the same is awkward and, worse, pollutes their corresponding namespace with entities that may be single use. Proposals such as N4560 present a complicated mechanism for providing tagged (self-documenting) tuple-like types, which may be necessary in some cases, but still represent a non-trivial amount of complexity that ideally should not be required. P0341 presents a similar idea with tagged parameter packs as return values.

The addition of decomposition declarations in C++17 in particular represents a significant step toward support of multiple return values as first class citizens. This feature, along with other ongoing efforts to add first class support for "product types" and other efforts such as P0341 show an encouraging movement away from the traditional `std::pair` and `std::tuple` towards comparable concepts without requiring the explicit types. (We expect, however, that the standard template library types will remain useful for algorithms where the identity of the elements is unimportant, while it *is* important to be able to name at least the outer, if not complete, type. In that respect, we hypothesize that we may in the future see the ability to construct a `std::tuple` from any tuple-like, as also suggested in P0197.)

On their own, however, these directions risk exacerbating the problem that this proposal aims to address. In particular, a

concern with the notion of returning a parameter pack, as presented in P0341, is that it opens a potential ABI can of worms. Moreover, parameter packs are not true types, and are subject to significant limitations, such as inability to make copies or pass them around as single entities, which are not shared by regular compound types. With generalized unpacking (P0535), product types and parameter packs are effectively interchangeable, and returning a product type rather than a parameter pack leverages existing ABI and techniques, rather than introducing something entirely novel. The remaining issue is one of naming; naming things — in this case, return types, especially for one-off functions — is *hard*.

It has been suggested on multiple occasions that the optimal solution to the above issues is to return an anonymous `struct`. This solves the problems of clutter and self-documentation, but runs afoul of a much worse issue; because the `struct` is *anonymous*, it can be difficult to impossible to give its name a second time in order to separate the declaration and definition of the function that wishes to use it. This, however, leads to a more interesting question: **why is it necessary to repeat the return value at all?**

Even in the case of return types that can be named, it may be that repeating the type name is excessively verbose or otherwise undesirable. Some might even call this a violation of the Don't Repeat Yourself principle, similar to some of the issues that `auto` for variable declaration was introduced to solve. (On the flip side, one could see the ability to elide the return type as subject to many abuses, again in much the manner of `auto`. However, many language features can be abused; this should not prevent the addition of a feature that would provide an important benefit when used correctly.)

While it is already possible in simple cases to use `decltype` and a sample invocation of the function, this is needlessly verbose, and as the argument list grows longer, it can quickly become unwieldy.

While both these features have use on their own, they are nevertheless related, and we believe that presenting them together makes sense, and strengthens the case for each.

Proposal

We propose, first, to remove the restriction against (anonymous) types as return values:

```
struct { int id; double value; } foo() { ... }
```

We believe this can be accomplished largely by simply removing the prohibition in [dcl.fct]¶11.

Second, we propose the addition of `decltype(return)` to name — in a function signature — the return type of a previously declared function. This is consistent with recent changes to the language that have progressively relaxed the requirements for how return types are specified, and provides an optimal solution to the following problem:

```
// foo.h
struct { int id; double value; } foo();
```

How does one now provide an external definition for `foo()`? With our proposal, the solution is simple:

```
// foo.cpp
decltype(return) foo()
{
  ...
  return { id, value };
}
```

Naturally, "previous declared" here means a declaration having the same name and argument list. This, for example, would remain illegal:

```
int foo(int);
float foo(float);

decltype(return) foo(double input) // does not match any previous declaration
{
  ...
  return result;
}
```

The reasons to prohibit an anonymous struct defined as a return type have also been significantly mitigated. Constructing the return result is a non-issue, since the type name may now be elided, and the combination of `auto` variable declarations, `decltype`, and the proposed mechanism for naming the return type in a function signature permit implicit naming of the type where necessary. In short, the prohibition ([dcl.fct]¶11) against defining types in return type specifications has become largely an artificial and arbitrary restriction which we propose to remove.

We additionally note that this prohibition is already not enforced by at least one major compiler (MSVC), and is enforced sporadically in others (see What about defining types in function pointer types?).

Interactions

Definition of a class-type as a return value type is currently ill-formed (although not universally enforced by existing major compilers), and the token sequence `decltype(return)` is currently ill-formed. Accordingly, this change will not affect existing and conforming code, and may cause existing but non-conforming code to become conforming. This proposal does not make any changes to other existing language or library features; while conceivable that some library methods might benefit from the feature, such changes are potentially breaking, and no such changes are proposed at this time.

Implementation and Existing Practice

The proposed feature to allow defining types (including anonymous types) during return value specification is already at least partly implemented by MSVC and (to a lesser extent) GCC and ICC, and is also partly conforming to C++14. The trick shown in Can't we do this already? as well as the curious, partial support in GCC and ICC (see What about defining types in function pointer types?) suggests that the existing prohibition may already be largely artificial, and that removing it would accordingly be a simple matter.

The proposed feature to allow `decltype(return)` to name the return value has not, to our knowledge, been implemented, but given that compilers must already compare the return value when confronted with an initial declaration followed by subsequent redeclarations and/or a definition, we do not anticipate any implementation difficulties.

Discussion

Can't we do this already?

Astute observers may note that this is already legal (as of C++14):

```
auto f()
{
  struct { int x, y; } result;
  // set values of result
  return result;
}
```

The critical problem with this, which we wish specifically to address, is that a (useful) forward declaration of such a function is not possible. We would see this as further justification for relaxing the existing prohibition, as proposed. (By "useful", we mean particularly a forward declaration that allows the function to be called without a definition being seen, which is required to use the function across translation units without the function being defined in each.)

Should we allow *named* types defined as return types?

Allowing both named and anonymous types is a logical consequence of simply lifting the existing [dcl.fct]¶11 prohibition as it is currently stated. It is also consistent, and already supported by MSVC:

```
// Equivalent to struct S { ... }; S foo();
struct S { ... } foo();
```

That said, the value here is less obvious, and we would find it acceptable to permit definition of only anonymous types as return types.

Isn't template parsing difficult?

Arthur O'Dwyer pointed out this interesting example:

```
template<class T>
struct {
    size_t s;
} // Declaring a templated type, right?
what_size(T t) {
    return {sizeof(t)};
}
```

It isn't obvious to the compiler, and not especially obvious to readers either, that this is a declaration of a templated function returning an anonymous type. Moreover, while the type itself is not templated, per-se, in effect it is, because (presumably?) each different instantiation of the function will have a distinct return type.

Since the primary motivation for this feature is for forward declarations of functions (per previous question, returning anonymous types is already possible with deduced return type), there are fewer use cases for the feature in conjunction with templated functions. As such, an easy cop-out is to retain the prohibition in these cases; we can always decide to lift it later.

An alternative (which may be worth considering for all cases) is to permit anonymous types only in trailing return type specifications, as follows:

```
auto foo -> struct { ... };
template<...> auto bar -> struct { ... };
```

Is `decltype(return)` dangerous?

P0224 previously recommended overloading `auto` as a mechanism for implicitly naming the return type given a prior declaration. While we believe this approach is feasible, there were some potential issues, which are discussed in P0224. While we would happily accept the solution proposed by P0224, we feel that `decltype(return)` is less ambiguous, both to readers and to compilers. It is slightly more verbose than `auto`, but not so much that we feel the added verbosity is an issue in those cases where we expect it to be used, and the extra verbosity may serve to deter "frivolous" use. Particularly, there is a clear distinction between inferred return values (the traditional use of `auto` as a return type) and "implied" return values (that is, the use of `decltype(return)` as an alternate spelling of a previously declared return type), which entirely avoids the issue this question, as it appears in P0224, addressed.

What about defining types in function pointer types?

An obvious consequence of relaxing [dcl.fct]¶11 is the desire to permit function pointers which return an anonymous struct. For example:

```
// Declare a function pointer type which returns an anonymous struct
using ReturnsAnonymousStruct = struct { int result; } (*)();

// Define a function using the same
int bar(ReturnsAnonymousStruct f) { return ((*f)()).result; }

// Provide a mechanism to obtain the return type of a function
template <typename T> struct ReturnType;

template <typename T, typename... Args>
struct ReturnType<T (*)(Args...)>
{
    using result_t = T;
};

// Declare a function that is a ReturnsAnonymousStruct
ReturnType<ReturnsAnonymousStruct>::result_t foo() { return {0}; }

// Use the function
int main()
{
    return bar(&foo);
}
```

It is our opinion that the proposed changes are sufficient to allow the above. (In fact, this example is already accepted by both GCC and ICC, although it is rejected by clang per [dcl.fct]¶11.) Accordingly, we feel that this proposal should be understood as intending to allow the above example and that additional wording changes to specify this behavior are not required at this time.

What about defining types in parameter types?

An obvious follow-on question is, should we also lift the prohibition against types defined in parameter specifications? There have been suggestions floated to implement the much requested named parameters in something like this manner. However, there are significant (in our opinion) reasons to not address this, at least initially. First, it is widely contested that this is not an optimal solution to the problem (named parameters) in the first place. Second, it depends on named initializers, which have a behavior that may not match that desired for named parameters. Third, this proposal works largely because C++ forbids overloading on return type, which may be leveraged to eliminate any ambiguity as to the deduction of the actual type of `decltype(return)`. This is not the case for parameters; the ability to overload functions would make a similar change for parameters much more complicated.

While we do not wish to categorically rule out future changes in this direction, we feel that it is not appropriate for this proposal to attempt to address these issues.

What about "pass-through" of return values having equivalent types?

Another question that has come up is if something like this should be allowed:

```
struct { int result; } foo() { ... }
struct { int result; } bar()
{
  return foo();
}
```

Specifically, others have expressed an interest in treating layout-compatible types as equivalent (or at least, implicitly convertible), particularly in the context of return values as in the above example.

Under the current rules (plus relaxed [dcl.fct]¶11), these two definitions have different return types which are not convertible. It is our opinion that the rules making these types different are in fact correct and desirable, and this proposal specifically does *not* include any changes which would make the types compatible. That said, we note that P0535 provides a ready solution to this problem:

```
struct { int result; } bar()
{
  return { [:]foo()... };
}
```

Does this conflicts with future "true" multiple return values?

There has been some discussion of "true" multiple return values, in particular with respect to RVO and similar issues. In particular, some features proposed by P0341 are very much in this vein. A point that bears consideration is if moving down the path of using anonymous (or not) structs for multiple return values will "paint us into a corner" where future optimization potential is prematurely eliminated.

It is our hope that these issues can be addressed with existing compound types (which will have further reaching benefit). Moreover, as previously stated, the use of compound types for multiple return values uses existing techniques and is well understood, whereas introducing "first class" multiple return values introduces questions of ABI and other issues.

What about deduced return types?

The relaxation of [dcl.fct]¶11 is not intended to extend to deduction of new types via deduced return types. In light of P0329, we might imagine a further extension that would allow us to lift this restriction:

```
auto foo()
{
  return { .x = 3, .y = 2 }; // deduce: struct { int x, y; }
}
```

However, we have reservations about allowing this, and do not at this time propose that this example would be well-formed.

Future Directions

In the Discussion section above, we presented a utility for extracting the return type from a function pointer type. The facility as presented has significant limitations; namely, it does not work on member functions and the several variations (e.g. CV-qualification) which apply to the same. We do not here propose a standard library implementation of this facility, which presumably would cover these cases, however there is room to imagine that such a facility could be useful, especially if the proposals we present here are adopted. (David Krauss points out that `std::reference_wrapper` can be used to similar effect... on *some* compilers. However, imperfect portability and the disparity between intended function and use for this result suggest that this is not the optimal facility for the problem.)

Another consideration that seems likely to come up is if we should further simplify the syntax for returning multiple values (conceivably, this could apply to both anonymous structs and to `std::pair` / `std::tuple`). Some have suggested allowing that the `struct` keyword may be omitted. In light of P0151 and P0341, we can conceive that allowing the syntax `<int x, double y> foo()` might be interesting (in contrast to P0341, we would suggest that this be shorthand for `std::tuple` if the names are omitted). At this time, we prefer to focus on the feature here presented rather than risk overextending the reach of this proposal. However, if this proposal is accepted, it represents an obvious first step to considering such features in the future.

A final consideration is the extension of `decltype(return)` to allow use within a function body. At the time of writing, we are not aware of a proposal to do so, although the idea has been floated on numerous occasions. We would hope to see such an addition,

which can be orthogonal to this proposal, in the near future. (This also serves as an additional argument for using `decltype(return)` to name the return value rather than `auto`.)

Acknowledgments

We wish to thank everyone on the `std-proposals` forum, especially Bengt Gustafsson, Arthur O'Dwyer and R. "Tim" Song, for their valuable feedback and insights.

References

- N4618 Working Draft, Standard for Programming Language C++

  http://wg21.link/n4618

- N4560 Extensions for Ranges

  http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2015/n4560.pdf

- P0151 Proposal of Multi-Declarators (aka Structured Bindings)

  http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2015/p0151r0.pdf

- P0197 Default Tuple-like Access

  http://wg21.link/p0197

- P0222 Allowing Anonymous Structs as Return Values

  http://wg21.link/p0224

- P0224 Implicit Return Type

  http://wg21.link/p0224

- P0329 Designated Initializer Wording

  http://wg21.link/p0329

- P0341 Parameter Packs Outside of Templates

  http://wg21.link/p0341

- P0535 Generalized Unpacking and Parameter Pack Slicing

  http://wg21.link/p0535